Rapport technique

Assistant pour la recommandation d'évènements culturels

Puls-Events

Hishem HYANI

Data ingenieur

19 avril 2026

Table des matières

1 Introduction **1**

2 Architecture du système RAG **2**

3 Technologies utilisés **3**

4 Pipeline ETL : Collecte et prétraitement des données **3**

5 Implémentation du Chatbot RAG **3**

5.1 Chunking 4

5.2 Construction et gestion de la base vectorielle 4

5.3 Interface utilisateur avec Streamlit 4

6 Tests unitaires et jeu de données d'évaluation **4**

7 Conclusion **5**

1 Introduction

La plateforme web de Puls-Events permet actuellement aux utilisateurs de découvrir des événements culturels en s'appuyant principalement sur des filtres classiques (par période, lieu, type....). Bien que fonctionnelle, cette approche reste limitée.

Avec les progrès du deep learning, les modèles de traitement du langage naturel ont connu une forte évolution, permettant de passer de systèmes basiques à des réseaux neuronaux capables de comprendre et générer du texte. L'arrivée des "Transformers" marque un tournant en rendant possible l'entraînement de modèles de grande taille (LLM), capables d'apprendre des représentations linguistiques à partir de vastes corpus de données. Cependant, malgré leurs performances, ces LLM présentent certaines limites. Notamment le fait qu'ils reposent sur des connaissances apprises lors de l'entraînement, ce qui peut générer des informations obsolètes voir des hallucinations.

C'est dans ce contexte qu'émergent les architectures de type RAG (Retrieval-Augmented Generation), qui combinent la puissance de génération des LLM avec un mécanisme de recherche d'informations externes, afin d'améliorer la pertinence, la fiabilité et la mise à jour des réponses produites.

En vue d'améliorer l'expérience utilisateur, Puls-Events souhaite donc enrichir sa plateforme d'un chatbot intelligent basé sur une architecture RAG.

Cette mission s'articule sur trois niveaux :

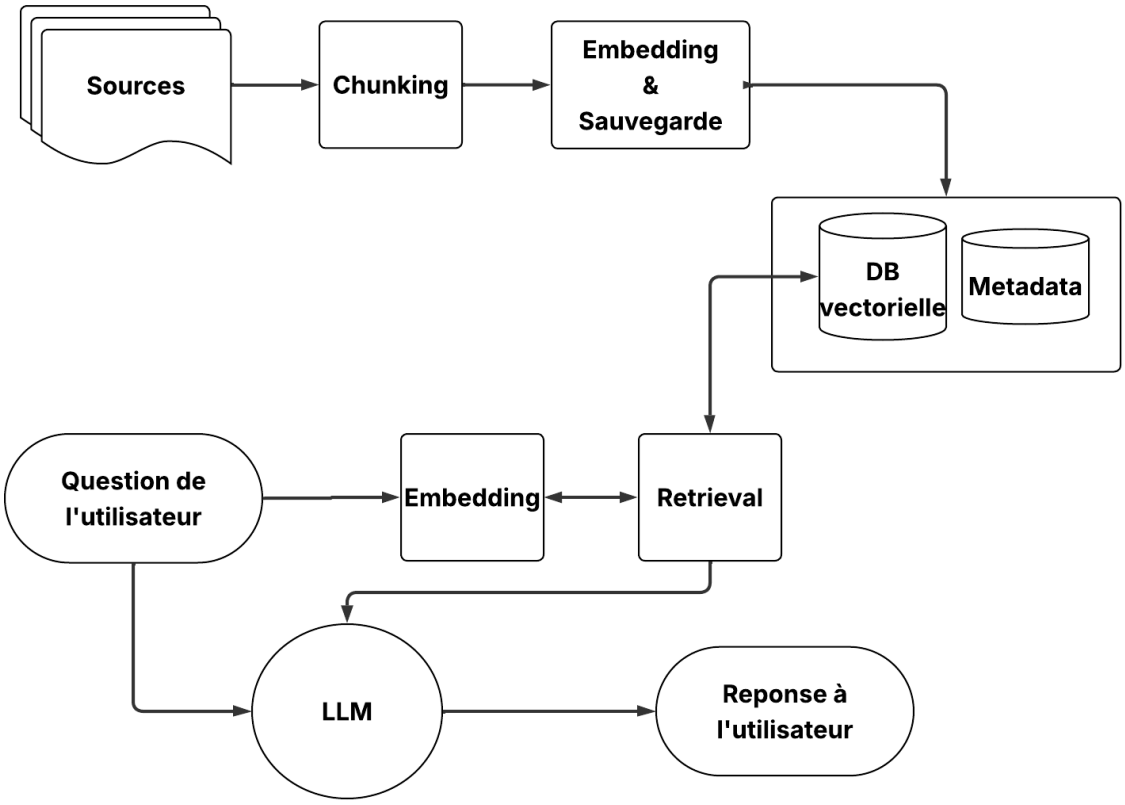
— La collecte des données et prétraitement

— L'implémentation d'un système RAG
— La mise en place d'une interface

2 Architecture du système RAG

Le schéma ci-dessous présente les différentes étapes d'une architecture RAG (Retrieval-Augmented Generation). Ce système peut être décomposé en deux pipelines distincts :

— le **pipeline de données**, responsable de la construction d'une **base vectorielle** (chunking, embedding, enrichissement de métadonnées et stockage).
— le **pipeline de génération**, qui améliore la qualité et la fiabilité de la réponse en réalisant une recherche sémantique (Retrieval) entre l'embedding de la question utilisateur et la base vectorielle. Les résultats les plus pertinents (top-k) sont sélectionnés, puis injectés dans le prompt du LLM pour produire une réponse contextuelle, précise et fidèle aux sources.



De nombreuses contraintes techniques et qualitatives doivent être prises en compte pour garantir une réponse pertinente. En particulier, il est indispensable de maîtriser la qualité des données en implémentant un pipeline ETL qui sera explicité plus bas.

3 Technologies utilisés

Cette section présente les principaux outils et bibliothèques utilisés pour la réalisation de ce Proof of Concept (POC).

Le gestionnaire de dépendances **Poetry** a été adopté afin d'assurer la reproductibilité du projet et la gestion des bibliothèques Python suivantes :

- **Requests** : interrogation de l'API externe.
- **Pandas** : nettoyage, transformation et structuration des données .
- **LangChain** : orchestration complète du pipeline RAG (chunking, embedding, vector store, retrieval et intégration du LLM) .
- **Mistral AI** : modèles `mistral-embed` pour l'embedding et `mistral-large-latest` pour la génération.
- **Streamlit** : interface utilisateur du chatbot.

4 Pipeline ETL : Collecte et prétraitement des données

Les données brutes sont collectées via l'API OpenDataSoft du jeu de données *evenements-publics-openagenda*. Le script Python permet d'interroger cette API avec une requête filtrée sur le département de l'Essonne et la date de début.

Les données sélectionnées incluent l'identifiant unique (`uid`), l'URL, les dates de début et de fin, le titre,la description en français, ainsi que les informations géographiques complètes (nom du lieu, adresse, code postal, ville et département).

Ces données récupérées au format JSON sont converties en DataFrame utile pour réaliser la phase de prétraitement :

- Nettoyage des valeurs manquantes (remplacées par des chaînes vides).
- Suppression des balises HTML et normalisation du texte à l'aide de **BeautifulSoup** et d'expressions régulières.
- Suppression des événements dont le titre ou la description est vide.
- Conversion des dates au format datetime avec le fuseau horaire Europe/Paris.
- Création d'un champ combiné `text_pour_embedding` concatenant le titre et la description longue, destiné à l'étape d'embedding ultérieure.

Les données prétraitées sont enfin sauvegardées au format Parquet, garantissant à la fois performance et conservation du type des données.

5 Implémentation du Chatbot RAG

Après la phase de collecte et de prétraitement des données, nous avons implémenté le système RAG complet pour alimenter le chatbot. Cette section se concentre particulièrement sur la construction de la base vectorielle, élément central de la performance du système.

5.1 Chunking

A partir des données prétraitées (stockées au format Parquet), le champ combiné `text_pour_embedding` est découpées en chunks à l'aide du textttRecursiveCharacterTextSplitter de LangChain.

Les paramètres retenus sont les suivants :

— **Taille de chunk** : 600 caractères

— **Chevauchement (overlap)** : 100 caractères

Chaque événement est transformé en Document LangChain, enrichis d'un ensemble de métadonnées (uid, titre, url, dates, localisation complète, index de chunk, etc.). Cette approche garantit à la fois une granularité fine et une bonne conservation du contexte.

5.2 Construction et gestion de la base vectorielle

Les chunks générés sont ensuite vectorisés puis stockés dans une base vectorielle locale.

Nous utilisons :

— **Le modèle d'embedding** : `mistral-embed` via l'API Mistral AI.

— **Une base vectorielle** : **FAISS**, choisie pour sa légèreté, sa rapidité.

Cette base vectorielle contient à la fois les vecteurs et les métadonnées associées, ce qui permet un retrieval précis et la possibilité de filtrage ultérieur (par date, par ville, par département, etc).

5.3 Interface utilisateur avec Streamlit

L'interface du chatbot a été développée avec **Streamlit**, permettant un déploiement rapide et une expérience utilisateur simple et interactive.

Le pipeline RAG est orchestré avec LangChain à l'aide d'une chaîne personnalisée :

— Chargement de la base vectorielle.

— Récupération des chunks les plus pertinents via le retriever.

— Injection du contexte dans un prompt système clair et strict (Utilise uniquement le contexte pour répondre).

— Génération de la réponse par le LLM.

Cette approche permet un prototypage rapide tout en offrant une interface fonctionnelle et professionnelle pour ce Proof of Concept.

6 Tests unitaires et jeu de données d'évaluation

Dans la continuité des étapes de collecte, de prétraitement et d'implémentation du système RAG, il est nécessaire de mettre en place des mécanismes de validation afin de garantir la qualité des données et la pertinence des réponses générées. Les tests unitaires réalisé avec pytest ont pour objectif de vérifier l'intégrité de la base vectorielle construite.

Ces tests portent principalement sur les dates ainsi que la cohérence géographique des événements (restreints au département de l'Essonne). Comme le montre la figure suivante, ces tests sont exécuté avec sucés.

```
hisdat@hisdat-A8-MAX:~/Data_2025/Projet_11$ poetry run pytest -v -W ignore script/test.p
y
================================= test session starts =================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0 -- /home/hisdat/.cache/pypoe
try/virtualenvs/poc-rag-7nBiOfQp-py3.12/bin/python
cachedir: .pytest_cache
rootdir: /home/hisdat/Data_2025/Projet_11
configfile: pyproject.toml
plugins: anyio-4.12.1, langsmith-0.7.14
collected 1 item

script/test.py::test PASSED                                                      [100%]

================================== 1 passed in 0.49s ==================================
```

En complément de ces tests unitaires, un jeu de données d'évaluation a été conçu afin de simuler des requêtes utilisateurs réalistes.

Afin d'évaluer le comportement du chatbot face à des requêtes hors de son domaine de compétence (événements culturels en Essonne), les questions suivantes peuvent être utilisé :

— Propose-moi un événement culturel dans les Yvelines.
— Peux-tu me recommander un restaurant ?

7 Conclusion

Grâce à une architecture modulaire basée sur LangChain, Mistral AI et FAISS, nous avons développé un chatbot capable de répondre de manière contextuelle aux questions sur les événements culturels dans l'Essonne.

Malgré ces résultats, plusieurs axes restent à explorer pour passer d'un POC à une solution de production. L'utilisation d'une base vectorielle non local, la gestion des coût de l'API mistral et l'implémentation d'un filtrage par métadonnées en amont de la recherche sémantique sont des pistes d'améliorations possibles .

Ce POC démontre néamoins la faisabilité technique et l'utilité d'un système RAG.